2026-06-10-C-helixagent-exposure

Workstream C — HelixAgent Exposure Extension (Analysis)

Type: Task / read-only analysis (PLANNING phase) **Date:** 2026-06-10 **Author:** Workstream-C analysis subagent (read-only; no code changed) **Scope root:** `/Volumes/T7/Projects/HelixCode` **Primary subject:** `submodules/helix_agent` (module `dev.helix.agent`, `go 1.26`)

> Evidence discipline: every claim below cites a real `file:line`. Paths are absolute from the repo root. Items not found are marked **ABSENT** explicitly. No fabrication.

---

1. Package map — `submodules/helix_agent/internal`

Note up-front: the task named `ensemble, llm, router, verifier, providers, modelsdev, models, clis`. The standalone `internal/providers` package is **ABSENT**; provider *implementations* live under `internal/llm/providers/` (48 provider dirs). The ensemble logic is split between `internal/llm` (core `RunEnsemble`) and `internal/ensemble` (multi-instance scaling). Each is mapped below.

1.1 `internal/llm` — provider abstraction + ensemble core

The package doc declares the central contract and the ensemble model (`submodules/helix_agent/internal/llm/doc.go:1-89`). The interface every provider satisfies is `LLMProvider` (`submodules/helix_agent/internal/llm/provider.go`, interface body):

```
Complete(ctx, *models.LLMRequest) (*models.LLMResponse, error)
CompleteStream(ctx, *models.LLMRequest) (<-chan *models.LLMResponse, error
HealthCheck() error
GetCapabilities() *models.ProviderCapabilities
ValidateConfig(map[string]interface{}) (bool, []string)
```

KEY FINDING (load-bearing): the interface has **no** `GetModels()` **method**. A provider's model list is surfaced only via `GetCapabilities().SupportedModels` (`submodules/helix_agent/internal/models/types.go:181-199`, field `SupportedModels []string`). This is the single seam through which "this provider's models" is read everywhere downstream (router, discovery fallback, etc.). Core ensemble executor: `RunEnsembleWithProviders` (`submodules/helix_agent/internal/llm/ensemble.go:55-112`) — fan-out to N providers under a `semaphore`, pick highest `Confidence`. Exported helpers `SetMaxConcurrentProviders` / `GetMaxConcurrentProviders` (`ensemble.go:26-37`).

1.2 `internal/ensemble` — multi-instance ensemble scaling

Sub-packages only (no top-level `.go`): `background/worker_pool.go`, `multi_instance/{coordinator,health_monitor,load_balancer}.go`, `synchronization/manager.go`. This is the horizontal-scaling layer (worker pool + coordinator + load balancer) that runs ensemble work across instances; it is orthogonal to the per-request `internal/llm` ensemble executor. Key types: `multi_instance.Coordinator`, `background.WorkerPool`.

1.3 `internal/router` — HTTP exposure surface (Gin)

`internal/router/router.go` (1639 lines) registers every public route via Gin (`router.go:471` onward). `internal/router/gin_router.go` (220 lines) is the constructor wrapper; `quic_server.go` adds a QUIC/HTTP3 transport. **This is the exposure surface** (full enumeration in §2). Key exported symbol: the route-setup function that wires every handler group.

1.4 `internal/verifier` — model/provider verification + discovery (LLMsVerifier)

The authoritative source for verification + scoring + discovery (CONST-036/040). - `ModelDiscoveryService` (`internal/verifier/discovery.go:23`) — discovers models per provider credential, scores, and selects the ensemble set. `Start(credentials)` (`discovery.go:107`), `GetSelectedModels() []*SelectedModel` (`discovery.go:447`), `GetDiscoveredModels() []*DiscoveredModel` (`discovery.go:452`). - `DiscoveredModel` (`discovery.go:50-63`): `ModelID`, `ModelName`, `Provider`, `ProviderID`, `Verified`, `CodeVisible`, `OverallScore`, `ScoreSuffix`, `Capabilities`, `ContextWindow`. - `SelectedModel` (`discovery.go:66-72`): embeds `*DiscoveredModel` + `Rank`, `VoteWeight`, `Selected`, `SelectedAt` — this is the AI-debate ensemble member shape. - `service.go` (1097 lines) — the verifier service; `scoring.go` / `enhanced_scoring.go` — scoring; `subscription_detector.go` — subscription/API detection; `provider_access.go` (see 1.4.1).

1.4.1 `internal/verifier/provider_access.go`

Static provider-access registry: `GetProviderAccessConfig(type)` (`:370`), `GetAllProviderAccessConfigs()` (`:379`), `GetProvidersWithSubscriptionAPI()` (`:384`), `GetProvidersWithRateLimitHeaders()` (`:395`). Describes how each provider type is reached (subscription API vs rate-limit headers).

1.5 `internal/llm/providers` — the 48 concrete providers

KEY FINDING: 48 provider implementation directories (`ls internal/llm/providers` → ai21, anthropic, anthropic_cu, azure, cerebras, chutes, claude, cloudflare, codestral, cohere, deepseek, fireworks, gemini, generic, githubmodels, groq, helixllm, huggingface, hyperbolic, junie, kilo, kimi, kimicode, lmstudio, mistral, modal, nia, nlpcloud, novita, nvidia, ollama, openai, openrouter, perplexity, publicai, qwen, replicate, sambanova, sarvam, siliconflow, together, upstage, venice, vertex, vulavula, xai, zai, zen, zhipu). All implement `LLMProvider` over **plain** `net/http` — e.g. `azure/azure.go` imports only `net/http` (no `azcore`), confirmed by its import block (no vendor SDK). `helixllm/provider.go:1-31` is the

HelixLLM submodule bridge (OpenAI-compatible, endpoints `/v1/chat/completions`, `/v1/models`, `/v1/embeddings`, default `https://localhost:8443`).

1.6 `internal/modelsdev` — models.dev catalog client

External catalog of model metadata. `Service` (`internal/modelsdev/service.go:13-23`) wraps `Client` + `Cache` with background refresh. `Client` hits `https://api.models.dev/v1` (`internal/modelsdev/client.go:14 DefaultBaseURL`). Rich `ModelInfo` shape (`internal/modelsdev/models.go:19-35`): pricing, capabilities, performance/benchmarks, family, tags. `ListModelsOptions` supports `Provider/Search/ModelType/Capability/Page/Limit` filters (`models.go:10-17`). This is **reference catalog metadata**, distinct from runtime provider model lists.

1.7 `internal/models` — shared DTOs

`ProviderCapabilities` (`types.go:181`), `LLMRequest`/`LLMResponse`, `EnsembleConfig`, `ModelParameters`, `Message`. The cross-package contract types used by router + handlers + providers.

1.8 `internal/clis` — managed CLI-agent pool (NOT LLM providers)

Manages external coding CLIs (aider, claude_code, codex, cline, openhands, kiro, continue, goose, plandex, cursor, windsurf, devin, … — `clis/types.go:18-40`+) as poolable instances (`pool.go`, `instance_manager.go`, `event_bus.go`). `CLIAgentType` string enum. **Orthogonal to the provider/model exposure ask** — this is about driving third-party agent binaries, not exposing LLM endpoints. Mentioned for completeness since the task listed it.

---

2. Current exposure surface — how providers/models/ensemble/HelixLLM reach users

Exposure surface = the Gin routes registered in `submodules/helix_agent/internal/router/router.go`. All under `/v1` (most behind `auth.Middleware`, group opened at `router.go:617/638`). Enumerated by evidence:

Concern	Route	Handler
Ensemble (AI debate)	`POST /v1/ensemble/completions`	inline closure → `providerRegistry.GetEnsembleService().RunEns`
Ensemble member list	`GET /v1/discovery/ensemble`	`discoveryHandler.GetEnsembleModels`
Debate model lookup	`GET /v1/discovery/debate-model`	`discoveryHandler.GetModelForDebate`
Provider list	`GET /v1/providers`	inline closure → `providerRegistry.ListProviders()` `GetCapabilities()`
		`providerMgmtHandler.*`

Concern	Route	Handler
Provider CRUD	`POST/GET/PUT/DELETE /v1/providers[/:id]`	
Provider verification	`GET /v1/providers/verification`, `POST /verify`, `GET /:id/verification`	`providerMgmtHandler.*`
Provider discovery	`GET /v1/providers/discovery`, `POST /discover`, `/rediscover`, `GET /best`	`providerMgmtHandler.*`
Model discovery	`GET /v1/discovery/models`, `/models/selected`, `/stats`, `POST /trigger`	`discoveryHandler.*`
Model metadata (catalog)	`GET /v1/models/metadata[...]` (+ `/compare`, `/capability/:cap`,`/:id/benchmarks`)	`modelMetadataHandler.*`
Completion model list	`GET /v1/completion/models`	`completionHandler.Models`
Verification (LLMsVerifier)	`GET /v1/verification/models`, `/status`, `POST /model` …	`verificationHandler.*`
Scoring	`GET /v1/scoring/top`, `/range`, `/model/:id` …	`scoringHandler.*`
Health (per provider)	`GET /v1/health/providers[...]`	`healthHandler.*`
Plain completion	`POST /v1/completion`, `/stream`, `/chat`, `/chat/stream`	`completionHandler.*`

How a user currently picks a model

- **Free-text `model` string** in the request body. `CompletionRequest.Model` (`internal/handlers/completion.go:28`) and the ensemble closure

(`router.go:702` → `ModelParameters.Model: req.Model`). No enum, no server-side validation against a catalog at call time.
- **HelixLLM is NOT a top-level selectable surface.** It is exposed *only* as one provider named `"helixllm"` inside the provider registry (default config `internal/services/provider_registry.go:749-764`, model id `helixllm-default`), enabled via `USE_HELIX_LLM=true`. There is no `/v1/helixllm/...` root and no first-class "HelixLLM" listing alongside the ensemble.
- **Is there a registry/catalog?** Yes — `services.ProviderRegistry` (`internal/services/provider_registry.go:82`) is the runtime registry. It registers 6 default providers (`registerDefaultProviders`, `:696-801`: deepseek, claude, gemini, helixllm, qwen, openrouter — each *disabled* until an API key is present). `ListProviders()` (`:963`), `ListProvidersOrderedByScore()` (`:1029`), `GetProvider(name)` (`:901`), `GetEnsembleService()` (`:1066`). The verifier (`ModelDiscoveryService`) is the authoritative discovery/scoring catalog; the models.dev `Service` is the external reference catalog.

Bluff/consistency flags found while mapping (worth a follow-up ticket)

- `GET /v1/completion/models` **returns a HARDCODED 3-model list** (`internal/handlers/completion.go:406+`: deepseek-coder, claude-3-sonnet-20240229, gemini-pro — literal maps, `time.Now()` timestamps). This contradicts CONST-036/037 and the resolved BLUFF-002 pattern (model lists must come from `providerManager.GetProviders()` / the verifier). **Flag for remediation**, not part of C's build-out but directly adjacent.
- `GET /v1/providers` correctly reads live `GetCapabilities().SupportedModels` (`router.go:783`) — this is the good pattern to extend.

---

3. GAP + proposed naming scheme

3.1 The gap

The request wants, **under ONE root**, individually exposed: (a) the AI-debate ensemble, (b) HelixLLM (when enabled), (c) **every discovered provider individually**, and (d) **each provider's WORKING (verified) models** — yielding dozens–hundreds of exposed items. Today:

1. The **ensemble** is exposed (`POST /v1/ensemble/completions`, `GET /v1/discovery/ensemble`) but as a single aggregate, not as a sibling in a unified catalog of selectable "targets".
2. **HelixLLM** is buried as provider `"helixllm"`; no first-class root entry.
3. **Providers** are individually listable (`GET /v1/providers`) but there is **no unified "catalog" that places ensemble + HelixLLM + each provider + each working-model as addressable, uniformly-named selection targets**.
4. **Per-provider working models**: only `GetCapabilities().SupportedModels` (static, not "working/verified") at `/v1/providers`; verified/working status lives separately in the verifier (`GetSelectedModels`/`GetDiscoveredModels`, `Verified` flag) and is surfaced only under `/v1/discovery/*` and `/v1/verification/*`. The two are **not joined into one selectable namespace**. There is no single endpoint returning "ensemble + helixllm + provider/model for every verified model".

Net: the building blocks all exist (registry, verifier discovery with `Verified` flag + score, ensemble service, HelixLLM provider). What is missing is a **unified catalog/selection namespace** that (i) lists ensemble + HelixLLM + every provider + every provider's *verified* models as uniformly-named entries, and (ii) lets a user target any one of them with the same `model` selector used today.

3.2 Proposed naming — consistent with what exists

Existing conventions observed: - Provider names are **lowercase identifiers**: `deepseek`, `claude`, `gemini`, `helixllm`, `qwen`, `openrouter` (`provider_registry.go:696-799`). - Models are referenced **bare or** `vendor/model`: `deepseek-coder`, `claude-3-sonnet-20240229`, and crucially `x-ai/grok-4` (`provider_registry.go:791`) — already a slash-namespaced id. - Discovery/ensemble responses key on `(Provider, ModelID)` pairs (`discovery_handler.go:85-95`, `DiscoveredModel.Provider/.ModelID`). - Ensemble object type is already string-tagged: `"object": "ensemble.completion"` (`router.go:740`).

Proposed unified selector grammar (extends, does not break, the free-text `model` field) — `provider/model` is the spine, with two reserved roots:

Exposed item	Proposed canonical name (selector)	Derived from
AI-debate ensemble (aggregate)	`ensemble` (alias `ensemble/auto`)	matches existing `ensemble.completion` object tag (`router.go:740`)
A named ensemble preset	`ensemble/<preset>` e.g. `ensemble/confidence_weighted`	mirrors `EnsembleConfig.Strategy` (`router.go:686`)
HelixLLM (whole provider)	`helixllm`	provider name `helixllm` (`provider_registry.go:753`)
A specific HelixLLM model	`helixllm/<model>` e.g. `helixllm/helixllm-default`	model id (`provider_registry.go:757`)
Any provider (whole)	`<provider>` e.g. `openai`, `anthropic`, `groq`	the 48 provider dir names + registry names
A provider's working model	`<provider>/<model_id>` e.g. `anthropic/claude-3-sonnet-20240229`, `openrouter/x-ai/grok-4`	`(Provider, ModelID)` pairs; preserves already-namespaced ids

Rationale for consistency: this reuses (1) the existing lowercase provider names, (2) the already-present `vendor/model` slash form (`x-ai/grok-4`), (3) the existing `ensemble.*` object tag, and (4) the `(Provider, ModelID)` join the discovery handler already emits. The catalog endpoint should return, per item: `name` (selector above), `kind` ∈ {`ensemble`, `provider`, `model`}, `provider`, `verified` bool + `overall_score` (from `DiscoveredModel.Verified/.OverallScore`, `discovery.go:50-63`), and

`enabled`. "WORKING models" = filter on `DiscoveredModel.Verified == true` (the verifier's authoritative flag) — never the static `SupportedModels` list, and never a hardcoded list (avoid the `completion.go:406` anti-pattern).

Suggested single new root (illustrative, to confirm in design): `GET /v1/catalog` returning the unified list, with the existing `model` request field accepting any `name` from it. This keeps ONE root over ensemble + HelixLLM + providers + verified models while leaving all current endpoints intact.

---

4. Provider / API SDK currency

KEY FINDING: `submodules/helix_agent` (module `dev.helix.agent`, `go 1.26`, `go.mod:1-3`) contains **NO cloud-provider or LLM-vendor SDKs at all** — grep for `aws|azure|openai|anthropic|vertex|genai|cohere|mistral|sashabaranov|bedrock` over `submodules/helix_agent/go.mod` returns **nothing**. Every one of the 48 providers is hand-rolled over `net/http` (confirmed: `azure/azure.go` imports only `net/http`). So for helix_agent there are **no provider SDK versions to age out** — currency risk there is in the *hand-written HTTP clients vs each vendor's latest API*, not in `go.mod`.

The SDKs named in the project tech-stack live in the **inner Go app** `/Volumes/T7/Projects/HelixCode/helix_code/go.mod` (module `dev.helix.code`, `go 1.26`). Observed versions (cite `helix_code/go.mod:<line>`):

Dependency	Version	go.mod line	Staleness flag (for later deep web research — NOT fetched here)
`github.com/Azure/azure-sdk-for-go/sdk/azcore`	`v1.16.0`	`:12`	**REVIEW** — azcore has shipped past v1.16; confirm latest
`github.com/Azure/azure-sdk-for-go/sdk/azidentity`	`v1.8.0`	`:13`	**REVIEW** — azidentity moves fast (auth/security); confirm latest
`github.com/aws/aws-sdk-go-v2`	`v1.32.7`	`:15`	**REVIEW** — core AWS SDK releases frequently; confirm latest
`github.com/aws/aws-sdk-go-v2/config`	`v1.28.7`	`:16`	**REVIEW**
`github.com/aws/aws-sdk-go-v2/credentials`	`v1.17.48`	`:17`	**REVIEW**
`github.com/aws/aws-sdk-`	`v1.23.1`	`:18`	**REVIEW (high value)** — Bedrock adds models/params

Dependency	Version	go.mod line	Staleness flag (for later deep web research — NOT fetched here)
`go-v2/service/bedrockruntime`			often; new model access likely gated on a newer version
`github.com/getzep/zep-go/v3`	`v3.10.0`	`:26`	**REVIEW** — confirm latest v3.x
`github.com/smacker/go-tree-sitter`	`v0.0.0-20240827…` (Aug 2024 pseudo-version)	`:39`	**STALE-LIKELY** — pinned to an Aug-2024 commit; tree-sitter grammars churn; confirm newer pseudo-version (same pin also in `helix_agent/go.mod:73`)
`github.com/gin-gonic/gin`	`v1.11.0`	`:27`	minor — CLAUDE stack says v1.11.0; helix_agent uses **v1.12.0** (`helix_agent/go.mod:5`) → **version skew between the two modules, flag**
`github.com/golang-jwt/jwt/v4`	`v4.5.2`	`:28`	acceptable (security-patched line); note v5 also pulled indirectly (`:127`)
`github.com/jackc/pgx/v5`	`v5.7.6`	`:32`	minor — newer v5.x exists; confirm
`github.com/redis/go-redis/v9`	`v9.17.2`	`:37`	minor — newer v9.x exists; confirm

No OpenAI / Anthropic / Gemini / Cohere / Mistral Go SDK appears in **either** `go.mod` (both use hand-written HTTP) — so "update the vendor SDK" does **not** apply to those; their currency risk is API-drift in the hand-rolled clients, which a deep web research pass against each vendor's current API reference should check.

> All "REVIEW/STALE" tags above are **flags for a later deep-web-research task** (§11.4.99 latest-source). This analysis did **not** fetch any registry/network to determine the true latest versions.

Executive summary (10 lines)

1. **Exposure surface** = `submodules/helix_agent/internal/router/router.go` (Gin, all under `/v1`); the unified catalog work belongs here + a new handler.
2. The **ensemble** is exposed at `POST /v1/ensemble/completions` (`router.go:676`, calls `GetEnsembleService().RunEnsemble`) and `GET /v1/discovery/ensemble`.
3. **Providers** are listable at `GET /v1/providers` (`router.go:773`, reads live `GetCapabilities().SupportedModels`); runtime registry is `services.ProviderRegistry` (`provider_registry.go:82`) with 48 provider impls.
4. **HelixLLM** is NOT a first-class root — only provider `"helixllm"` (`provider_registry.go:749-764`, gated by `USE_HELIX_LLM=true`).
5. **Working/verified models** live in the verifier (`ModelDiscoveryService`, `discovery.go:23`; `DiscoveredModel.Verified` flag `discovery.go:50`), surfaced only under `/v1/discovery/*` and `/v1/verification/*`.
6. **Model selection today** = free-text `model` string (`completion.go:28`); no unified selectable namespace joining ensemble + HelixLLM + provider + verified model.
7. **GAP:** build a unified catalog/selection namespace under one root that lists all four item classes as uniformly-named, addressable targets.
8. **Proposed naming:** `ensemble` / `ensemble/<preset>`, `helixllm[/<model>]`, `<provider>`, `<provider>/<model_id>` — reuses existing lowercase provider names + the already-present `vendor/model` slash form (`x-ai/grok-4`, `provider_registry.go:791`); "working" = `DiscoveredModel.Verified == true`.
9. **Bluff flag:** `GET /v1/completion/models` returns a **hardcoded 3-model list** (`completion.go:406`) — contradicts CONST-036/BLUFF-002; remediate.
10. **SDK currency:** helix_agent has **zero vendor SDKs** (hand-rolled HTTP); cloud SDKs live in `helix_code/go.mod` — flag aws bedrockruntime v1.23.1, azcore v1.16.0, azidentity v1.8.0, go-tree-sitter (Aug-2024 pin), gin skew (1.11.0 vs 1.12.0) for later deep-web-research version checks.

Exposure surface location: `submodules/helix_agent/internal/router/router.go` (route registration; ensemble at `:676`, providers at `:773`, discovery at `:1306-1313`, verification at `:1378-1387`). Runtime registry: `submodules/helix_agent/internal/services/provider_registry.go`.

Proposed naming examples: `ensemble`, `ensemble/confidence_weighted`, `helixllm`, `helixllm/helixllm-default`, `anthropic/claude-3-sonnet-20240229`, `openrouter/x-ai/grok-4`, `groq`, `deepseek/deepseek-coder`.